

Programmazione II (a.a. 2025/2026)

M. Pasqua

Appello 4 del 17 luglio 2026

Tempo a disposizione: 120 minuti

Punteggio massimo: 31 punti

Si crei un progetto Java contenente i package `it.univr` e `it.univr.plid`. Si copino al loro interno le classi del compito. Non si modifichino le dichiarazioni dei metodi, dei campi e delle classi, se non *espressamente richiesto*. Si possono definire altri campi e metodi, ma devono essere `private`. Si possono aggiungere annotazioni ai metodi e importare classi di libreria.

Gli identificatori di un linguaggio di programmazione sono spesso scritti in *snake-style*, in cui le parole sono separate da underscore, ad esempio `snakes_Are_sloW` (il maiuscolo o il minuscolo è a libera scelta del programmatore). Si immagini un altro modo di scrivere gli identificatori, in cui le parole vengono prima troncate a massimo tre caratteri e poi concatenate, ad esempio `snaAreslo`. Questo modo alternativo di scrivere gli identificatori si chiamerà *three-style*. Entrambi sono esempi di identificatori *multiword*, cioè composti da più parole.

L'interfaccia (già fornita) che definisce un identificatore ha un unico metodo che permette di tradurre un identificare in stringa:

```
public interface Identifier {  
    String toString();  
}
```

Esercizio 1 (11 punti) Si completi la classe astratta `MultiWordIdentifier` che rappresenta un identificatore multiword ed implementa l'interfaccia `Identifier`. Tale classe prevede due metodi `toString`: il principale richiamerà il secondario su ciascuna parola componente l'identificatore e concatenerà il risultato in un'unica stringa. Il costruttore della classe riceve come parametri un elenco variabile di parole, che formeranno l'identificatore. Un identificatore multiword contiene parole formate da soli caratteri alfabetici. Nello specifico:

- Completare la dichiarazione della classe.
- Completare il costruttore della classe.
- Implementare il metodo `toString` principale (senza parametri).

Esercizio 2 (10 punti) Si completi la classe concreta `SnakeStyleIdentifier`, sotto-classe di `MultiWordIdentifier`, che modella identificatori snake-style. La classe ha due costruttori: il primo costruisce l'identificatore a partire da un elenco variabile di parole componenti, mentre il secondo a partire da un qualsiasi iterabile di parole componenti. La classe deve implementare correttamente il metodo `toString` secondario, per

generare componenti codificate in snake-style. La classe dispone inoltre di un metodo `toThreeStyle` che ritorna un identificatore contenente le parole dell'identificatore corrente codificate in three-style. Nello specifico:

- Completare i costruttori della classe.
- Implementare il metodo `toString` secondario (con parametri).
- Implementare il metodo `toThreeStyle`.

Esercizio 3 (10 punti) Si completi la classe concreta `ThreeStyleIdentifier`, sotto-classe di `MultiWordIdentifier`, che modella identificatori three-style. La classe ha due costruttori: il primo costruisce l'identificatore a partire da un elenco variabile di parole componenti, mentre il secondo concatenando le parole componenti di altri identificatori multiword passati come parametro. La classe deve implementare correttamente il metodo `toString` secondario, per generare componenti codificate in three-style. La classe dispone inoltre di un metodo `toSnakeStyle` che ritorna un identificatore contenente le parole dell'identificatore corrente codificate in snake-style. Nello specifico:

- Completare i costruttori della classe.
- Implementare il metodo `toString` (con parametri).
- Implementare il metodo `toSnakeStyle`.

Se tutto è corretto, l'esecuzione del metodo `main` della classe `MainPLid` già fornita stamperà a video qualcosa del tipo:

```
Parole iniziali: 1
ciao             2
Amico           3
comE            4
va              5
                6
id1 (three-style) = ciaAmicomva 7
id2 (snake-style) = ciao_Amico_comE_va 8
id3 (three-style) = ciaAmicomvaciaAmicomva 9
id3 (snake-style) = ciao_Amico_comE_va_ciao_Amico_comE_va 10
```